

HDOC – Day 28

C Programming Assignment

Name - Arushi Gusain

Sap id - 590033489

Batch 15

Date - 21/09/2026

Variable-Length 2D Arrays and Row-Major Order in C

Question 1

Task: Read positive rows and columns, create a variable-length 2D integer array, read elements column by column, and finally print the same array in row-major order using one loop and a linear index.

Algorithm

1. Start.
2. Read rows and columns. If either is not positive, repeat until both are positive.
3. Declare `int a[rows][cols]`.
4. Use the outer loop for columns and the inner loop for rows; read `a[i][j]`.
5. For $k = 0$ to $\text{rows} \times \text{cols} - 1$, compute $\text{row} = k / \text{cols}$ and $\text{column} = k \% \text{cols}$.
6. Print `a[row][column]`.
7. Stop.

C Program

```
#include <stdio.h>

int main(void)
{
    int rows, cols;
    do {
        printf("Enter number of rows and columns: ");
        scanf("%d %d", &rows, &cols);
        if (rows <= 0 || cols <= 0)
            printf("Both values must be positive. Try again.\n");
    } while (rows <= 0 || cols <= 0);
}
```

```

} while (rows <= 0 || cols <= 0);
int a[rows][cols];
printf("\nEnter the array elements column by column:\n");
for (int j = 0; j < cols; j++) {
    for (int i = 0; i < rows; i++) {
        printf("a[%d][%d] = ", i, j);
        scanf("%d", &a[i][j]);
    }
}
printf("\nElements in row-major order:\n");
for (int k = 0; k < rows * cols; k++) {
    int i = k / cols;
    int j = k % cols;
    printf("%d ", a[i][j]);
}
printf("\n");
return 0;
}

```

Test Cases

Case	Rows x cols	Input column wise	Expected row major output
1	2 x 3	1 4 2 5 3 6	1 2 3 4 5 6
2	3 x 2	10 30 50 20 40 60	10 20 30 40 50 60
3	0 x 3, then 2 x 2	invalid, then 1 3 2 4	1 2 3 4

Sample Execution (Question 1)

Enter number of rows and columns: 2 3

Enter the array elements column by column:

a[0][0] = 1

a[1][0] = 4

a[0][1] = 2

a[1][1] = 5

a[0][2] = 3

a[1][2] = 6

Elements in row-major order:

1 2 3 4 5 6

Output

```
Session Actions Edit View Help
(kali㉿kali)-[~]
$ cc hdoc28.c
(kali㉿kali)-[~]
$ ./a.out
Enter number of rows and columns: 2 3

Enter the array elements column by column:
a[0][0] = 1
a[1][0] = 4
a[0][1] = 2
a[1][1] = 5
a[0][2] = 3
a[1][2] = 6

Elements in row-major order:
1 2 3 4 5 6

(kali㉿kali)-[~]
$ ./a.out
Enter number of rows and columns: 3 2

Enter the array elements column by column:
a[0][0] = 10
a[1][0] = 20
a[2][0] = 30
a[0][1] = 40
a[1][1] = 50
a[2][1] = 60

Elements in row-major order:
10 40 20 50 30 60

(kali㉿kali)-[~]
$ ./a.out
Enter number of rows and columns: 0 3
Both values must be positive. Try again.
Enter number of rows and columns: 2 2
```

Question 2

Task: Read elements into a variable-length 2D array, copy them into a 1D array in row-major order, and display both arrays.

Algorithm

1. Start.
2. Read positive rows and columns; repeat for invalid dimensions.
3. Declare `int a[rows][cols]` and `int b[rows*cols]`.
4. Read the 2D array row by row.
5. Copy `a[i][j]` to `b[i*cols+j]`.
6. Display the 2D array.
7. Display the 1D array.
8. Stop.

C Program

```
#include <stdio.h>

int main(void)
{
    int rows, cols;
    do {
        printf("Enter number of rows and columns: ");
        scanf("%d %d", &rows, &cols);
        if (rows <= 0 || cols <= 0)
            printf("Both values must be positive. Try again.\n");
    } while (rows <= 0 || cols <= 0);
    int a[rows][cols];
    int b[rows * cols];
    printf("Enter %d elements row by row:\n", rows * cols);
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            scanf("%d", &a[i][j]);
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            b[i * cols + j] = a[i][j];
    printf("\n2D array:\n");
```

```

for (int i = 0; i < rows; i++) {
for (int j = 0; j < cols; j++)
printf("%d ", a[i][j]);
printf("\n");
}
printf("\n1D array in row-major order:\n");
for (int k = 0; k < rows * cols; k++)
printf("%d ", b[k]);
printf("\n");
return 0;
}

```

Test Cases

Case	Rows x cols	2D input(row-wise)	Expected 1D array
1	2 x 3	1 2 3 4 5 6	1 2 3 4 5 6
2	3 x 2	10 20 30 40 50 60	10 20 30 40 50 60
3	2 x 2	7 8 9 10	7 8 9 10

Sample Execution (Question 2)

Enter number of rows and columns: 2 3

Enter 6 elements row by row:

1 2 3 4 5 6

2D array:

1 2 3

4 5 6

1D array in row-major order:

1 2 3 4 5 6

Output

```
Session Actions Edit View Help
(kali㉿kali)-[~]
$ vi h28.c

(kali㉿kali)-[~]
$ cc h28.c

(kali㉿kali)-[~]
$ ./a.out
Enter number of rows and columns: 2 3
Enter 6 elements row by row:
1 2 3 4 5 6

2D array:
1 2 3
4 5 6

1D array in row-major order:
1 2 3 4 5 6

(kali㉿kali)-[~]
$ ./a.out
Enter number of rows and columns: 3 2
Enter 6 elements row by row:
10 20 30 40 50 60

2D array:
10 20
30 40
50 60

1D array in row-major order:
10 20 30 40 50 60
```